

Tối ưu hóa quy hoạch động

Đường Văn Bản

Nguồn gốc: Dynamic Programming Optimization

Dynamic Programming / DP Optimization - Tang Wenbin.pdf

Abstract

Nguồn là một bộ slide PowerPoint về tối ưu hóa quy hoạch động. Bản tiếng Việt này chuyển phần nội dung thuật toán thành ghi chú đồng hành, giữ các ví dụ và ý tưởng chính về giảm chi phí chuyển trạng thái, cấu trúc dữ liệu, hàng đợi đơn điệu và tính đơn điệu của quyết định.

Tư tưởng chung

Độ phức tạp thời gian của một lời giải quy hoạch động thường có thể tách thành hai phần:

số trạng thái $\times$ thời gian chuyển cho mỗi trạng thái.

Vì vậy, khi một quy hoạch động trực tiếp chưa đủ nhanh, ta thường đi theo một trong hai hướng:

- giảm số trạng thái cần xét;
- tăng hiệu quả của phép chuyển trạng thái.

Tài liệu gốc chủ yếu minh họa hướng thứ hai: giữ mô hình trạng thái tự nhiên, nhưng nhận ra phép cực trị trong công thức chuyển có thể được thay bằng truy vấn dữ liệu, cấu trúc đơn điệu, hàng đợi đơn điệu hoặc tính đơn điệu của quyết định.

1 Dãy con tăng dài nhất

Cho một dãy số nguyên A độ dài N . Cần tìm một dãy con dài nhất $x_1, \dots, x_k$ sao cho

$$1 \leq x_1 < x_2 < \dots < x_k \leq N \quad \text{và} \quad A[x_1] < A[x_2] < \dots < A[x_k].$$

Gọi $F[i]$ là độ dài dãy con tăng dài nhất kết thúc tại $A[i]$. Công thức cơ bản là

$$F[i] = \max\{F[j] \mid j < i, A[j] < A[i]\} + 1.$$

Thuật toán trực tiếp có $O(N)$ trạng thái và mỗi trạng thái kiểm tra $O(N)$ vị trí trước đó, nên tổng độ phức tạp là $O(N^2)$.

Tối ưu bằng cấu trúc dữ liệu

Nút thắt nằm ở phép lấy cực đại với điều kiện $A[j] < A[i]$. Sau khi đã xử lý các vị trí trước i , ta có một tập khóa dạng (f, x) , trong đó f là giá trị quy hoạch động và x là giá trị phần tử tương ứng. Với trạng thái i , cần trả lời truy vấn:

$$\max\{f \mid x < A[i]\}.$$

Sau đó thêm khóa mới $(F[i], A[i])$ vào tập.

Nếu miền giá trị của $A[i]$ có thể nén tọa độ, cây đoạn hoặc cây Fenwick cho phép thực hiện truy vấn tiền tố và cập nhật điểm trong $O(\log N)$. Khi đó tổng thời gian giảm từ $O(N^2)$ xuống $O(N \log N)$.

Tối ưu bằng loại bỏ khóa thừa

Một cách nhìn khác là duy trì chỉ các khóa còn có ích. Nếu tồn tại hai khóa (f_1, x_1) và (f_2, x_2) sao cho

$$f_1 \geq f_2, \quad x_1 \leq x_2,$$

thì (f_2, x_2) không bao giờ tốt hơn (f_1, x_1) cho bất kỳ truy vấn $x < A[i]$ nào, nên có thể bỏ nó. Sau khi loại bỏ toàn bộ khóa bị thống trị, danh sách còn lại có cả f và x tăng nghiêm ngặt. Truy vấn “khóa có x nhỏ hơn $A[i]$ và lớn nhất” có thể trả lời bằng tìm kiếm nhị phân. Khi thêm khóa mới, ta xóa các khóa bị nó thống trị để khôi phục bất biến.

Cách diễn giải này dẫn đến thuật toán kinh điển duy trì giá trị nhỏ nhất có thể của phần tử cuối cho mỗi độ dài dãy con tăng.

2 Dãy con Fibonacci: ZJU 2672

Một dãy $a_1, a_2, \dots, a_n$ được gọi là dãy Fibonacci nếu

$$a_i = a_{i-1} + a_{i-2} \quad (i \geq 3).$$

Với một dãy số nguyên cho trước, bài toán yêu cầu tìm dãy con Fibonacci dài nhất, với $N \leq 3000$.

Gọi $F[i, j]$ là độ dài dãy con Fibonacci dài nhất kết thúc bằng hai phần tử A_i, A_j . Với $i < j$, trạng thái tự nhiên có dạng

$$F[i, j] = \max\{F[k, i] + 1 \mid k < i, A_k + A_i = A_j\}.$$

Số trạng thái là $O(N^2)$, và nếu mỗi trạng thái quét mọi k thì tổng thời gian là $O(N^3)$.

Điểm quan trọng là với cặp (i, j) đã cố định, ta chỉ cần tìm vị trí k sao cho $A_k = A_j - A_i$. Nếu cần chọn trong nhiều vị trí có cùng giá trị, vị trí lớn nhất là lựa chọn có ý nghĩa nhất cho trạng thái kết thúc gần nhất. Có thể tìm k bằng:

- tìm kiếm nhị phân trong danh sách các vị trí của giá trị $A_j - A_i$, đạt $O(\log N)$ cho mỗi chuyển;
- hoặc duy trì bảng vị trí theo thứ tự quét để truy vấn trong $O(1)$ trong những biến thể có thể xử lý tuyến tính.

Như vậy phần chuyển không còn là phép quét $O(N)$.

3 Painting the Balls: SGU 183

Có N quả bóng trên một hàng, ban đầu đều màu trắng. Ta có thể chọn quả bóng thứ i để tô đen với chi phí C_i . Điều kiện là trong mọi đoạn liên tiếp gồm M quả bóng phải có ít nhất hai quả bóng đen. Hãy tìm

tổng chi phí nhỏ nhất. Ràng buộc điển hình là

$$2 \leq N \leq 10000, \quad 2 \leq M \leq 100, \quad M \leq N.$$

Trạng thái tự nhiên lưu hai quả bóng đen cuối cùng. Gọi $F[i, j]$ là chi phí nhỏ nhất khi hai quả bóng đen cuối cùng ở i và j với $i < j$. Khi chọn j làm quả bóng đen mới, quả bóng đen trước i là một vị trí k thỏa điều kiện không để xuất hiện đoạn dài M chỉ có ít hơn hai quả bóng đen:

$$F[i, j] = C_j + \min\{F[k, i] \mid j - M < k < i\}.$$

Số trạng thái là $O(NM)$, nhưng chuyển trực tiếp mất $O(M)$, cho tổng $O(NM^2)$.

Nhìn phép chuyển như truy vấn cực tiểu

Với i và j cố định, quyết định là giá trị nhỏ nhất trên một khoảng chỉ số k . Cây đoạn có thể giảm thời gian quyết định xuống $O(\log M)$.

Tuy nhiên còn có thể tốt hơn. Khi xử lý theo từng giai đoạn i và tính các j theo thứ tự giảm dần, các khoảng cần lấy cực tiểu trượt dần. Thông tin đã tính cho $F[i, j + 1]$ có thể được tái sử dụng cho $F[i, j]$. Duy trì cực tiểu của cửa sổ hiện tại bằng hàng đợi đơn điệu giúp mỗi chuyển còn $O(1)$, đưa tổng thời gian về $O(NM)$.

4 USACO DEC04 Divide

Trên một luống hoa độ dài L với $L \leq 10^6$, cần lắp các vòi phun. Bán kính mỗi vòi nằm trong đoạn $[A, B]$, với $1 \leq A \leq B \leq 10^3$, và mỗi vị trí phải được đúng một vòi che phủ. Ngoài ra có $N \leq 10^3$ đoạn cho trước; mọi bông hoa trong cùng một đoạn phải do cùng một vòi che phủ. Mục tiêu là dùng ít vòi nhất.

Gọi $F[i]$ là số vòi ít nhất khi vòi cuối cùng kết thúc ở vị trí i . Khi đặt vòi cuối che đoạn từ sau j đến i , độ dài đường kính phải nằm trong khoảng $[2A, 2B]$, đồng thời điểm cắt j không được nằm bên trong một đoạn bị ràng buộc. Công thức là

$$F[i] = 1 + \min\{F[j] \mid 2A \leq i - j \leq 2B, j \text{ không nằm trong đoạn cấm}\}.$$

Nếu j nằm trong một đoạn cho trước, ta có thể đặt $F[j] = +\infty$. Khi đó công thức trở thành truy vấn cực tiểu cửa sổ:

$$F[i] = 1 + \min\{F[j] \mid 2A \leq i - j \leq 2B\}.$$

Cây đoạn cho thời gian chuyển $O(\log L)$. Nhưng do cửa sổ chỉ trượt khi i tăng, ta có thể duy trì các ứng viên bằng hàng đợi đơn điệu: loại bỏ phần tử ra khỏi cửa sổ, và loại bỏ từ cuối hàng đợi các vị trí có giá trị F không tốt hơn vị trí mới. Khi đó mỗi vị trí vào và ra hàng đợi nhiều nhất một lần, cho tổng thời gian $O(L)$.

5 Bất đẳng thức tứ giác và đơn điệu quyết định

Xét các quy hoạch động hai chiều có trạng thái $F[i, j]$, trong đó quyết định k chia đoạn và phụ thuộc vào hai phần $F[i, k]$, $F[k, j]$. Ký hiệu quyết định tối ưu là $K[i, j]$. Nếu có thể chứng minh tính đơn điệu

$$K[i, j - 1] \leq K[i, j] \leq K[i + 1, j],$$

thì khi tính $F[i, j]$ ta không cần thử mọi k trong đoạn $[i, j]$. Tổng số lần liệt kê quyết định trên toàn bộ các trạng thái độ dài đoạn có thể giảm xuống $O(N^2)$.

Ví dụ: gộp đá

Bài toán gộp đá có dạng chuyển quen thuộc:

$$F[i, j] = \text{sum}(i, j) + \min_{i \leq k < j} \{F[i, k] + F[k + 1, j]\}.$$

Cách tính trực tiếp thử mọi k cho mỗi cặp (i, j) , nên mất $O(N^3)$. Khi hàm chi phí thỏa điều kiện để quyết định tối ưu đơn điệu, ta chỉ cần xét

$$K[i, j - 1] \leq k < K[i + 1, j],$$

và nhận được lời giải $O(N^2)$.

Kết luận

Các ví dụ trên đều bắt đầu từ cùng một câu hỏi: phép chuyển đang thật sự làm gì? Nếu nó chỉ là lấy cực đại hoặc cực tiểu trên một miền có cấu trúc, ta nên tìm cấu trúc dữ liệu hoặc bất biến đơn điệu phù hợp. Nếu quyết định tối ưu dịch chuyển có quy luật, hãy chứng minh tính đơn điệu của quyết định và thu hẹp phạm vi thử. Đây là tinh thần chính của tối ưu hóa quy hoạch động: không nhất thiết thay mô hình trạng thái, mà làm cho phép chuyển rẻ hơn.